

大模型知识编辑 (Knowledge Editing for LLMs)

学号: SC2616025 姓名: 郑慧敏

Task 1: 基础环境搭建与基线测试 (Baseline Evaluation)

1、构建数据集

构建了包含 10 条事实更新的数据集, 保存在 data\custom_facts.json 中。ground_truth 为过时的答案, target_new 为最新正确答案。还配有同义改写 rephrase_prompt 用于计算 PS, 以及一个领域相近但无关的 locality 问题, 用于计算 NS。数据集内容涵盖企业高管、政治变动、体育赛事、地理变更、品牌更名等真实世界知识更新。

```
EasyEdit > data > {} custom_facts.json > {} 5
1  [
2    {
3      "prompt": "推特(X)的现任CEO是",
4      "subject": "推特(X)",
5      "target_new": "琳达·亚卡里诺",
6      "ground_truth": "埃隆·马斯克",
7      "rephrase_prompt": "X平台(原推特)的首席执行官是谁?",
8      "locality_prompt": "苹果公司的现任CEO是",
9      "locality_ground_truth": "蒂姆·库克"
10   },
11   {
12     "prompt": "英国现任首相是",
13     "subject": "英国",
14     "target_new": "基尔·斯塔默",
15     "ground_truth": "鲍里斯·约翰逊",
16     "rephrase_prompt": "目前谁在担任英国首相?",
17     "locality_prompt": "法国现任总统是",
18     "locality_ground_truth": "埃马纽埃尔·马克龙"
19   },
20   {
21     "prompt": "Facebook的母公司现在叫",
22     "subject": "Facebook",
23     "target_new": "Meta Platforms",
24     "ground_truth": "Facebook Inc",
25     "rephrase_prompt": "拥有Facebook的公司的企业名称是什么?",
26     "locality_prompt": "Google的母公司是",
27     "locality_ground_truth": "Alphabet"
28   },
29   {
30     "prompt": "社交媒体平台Twitter正式更名为",
31     "subject": "Twitter",
32     "target_new": "X",
33     "ground_truth": "Twitter",
34     "rephrase_prompt": "取代Twitter品牌的新名称是什么?",
35     "locality_prompt": "Instagram归属于哪家公司",
36     "locality_ground_truth": "Meta"
37   },
38   ]
```

```

38     {
39         "prompt": "英国现任君主是",
40         "subject": "英国",
41         "target_new": "查尔斯三世国王",
42         "ground_truth": "伊丽莎白二世女王",
43         "rephrase_prompt": "目前谁在位担任英国君主? ",
44         "locality_prompt": "美国现任总统是",
45         "locality_ground_truth": "唐纳德·特朗普"
46     },
47     {
48         "prompt": "2022年国际足联世界杯的冠军是",
49         "subject": "2022年国际足联世界杯",
50         "target_new": "阿根廷",
51         "ground_truth": "法国",
52         "rephrase_prompt": "哪个国家队赢得了2022年世界杯冠军?",
53         "locality_prompt": "2018年国际足联世界杯的冠军是",
54         "locality_ground_truth": "法国"
55     },
56     {
57         "prompt": "印度尼西亚的首都是",
58         "subject": "印度尼西亚",
59         "target_new": "努桑塔拉",
60         "ground_truth": "雅加达",
61         "rephrase_prompt": "印度尼西亚的行政中心在哪里? ",
62         "locality_prompt": "泰国的首都是",
63         "locality_ground_truth": "曼谷"
64     },
65     {
66         "prompt": "亚马逊公司的CEO是",
67         "subject": "亚马逊",
68         "target_new": "安迪·贾西",
69         "ground_truth": "杰夫·贝索斯",
70         "rephrase_prompt": "谁领导亚马逊担任其首席执行官? ",
71         "locality_prompt": "微软公司的CEO是",
72         "locality_ground_truth": "萨提亚·纳德拉"
73     },
74     {

```

```

75         "prompt": "特斯拉总部位于",
76         "subject": "特斯拉",
77         "target_new": "得克萨斯州奥斯汀",
78         "ground_truth": "加利福尼亚州帕洛阿尔托",
79         "rephrase_prompt": "特斯拉的主要办公地点位于哪个城市和州? ",
80         "locality_prompt": "苹果公司总部位于",
81         "locality_ground_truth": "加利福尼亚州库比蒂诺"
82     },
83     {
84         "prompt": "微软Windows最新的主要版本是",
85         "subject": "微软Windows",
86         "target_new": "Windows 11",
87         "ground_truth": "Windows 10",
88         "rephrase_prompt": "Windows操作系统的最新主要发行版是什么?",
89         "locality_prompt": "macOS最新版本的代号是",
90         "locality_ground_truth": "Sequoia"
91     }
92 ]

```

2、记录模型在知识编辑前的回答

运行 baseline.py，得到以下结果：

```
(EasyEdit) PS Z:\EasyEdit> python FinalAssignment\baseline.py
GPU: NVIDIA GeForce RTX 5070
Z:\Anaconda3\envs\EasyEdit\lib\site-packages\timm\models\hub.py:4: FutureWarning: Importing from timm.models.hub is deprecated,
please import via timm.models
warnings.warn(f"Importing from {__name__} is deprecated, please import via timm.models", FutureWarning)
Loading Qwen2.5-0.5B model ...
```

```
[1] 问题：推特 (x) 的现任CEO是
输出：杰夫·雷蒙德
旧答案：埃隆·马斯克 新答案：琳达·亚卡里诺
匹配旧知识：否 匹配新知识：否

[2] 问题：英国现任首相是
输出：约翰逊
旧答案：鲍里斯·约翰逊 新答案：基尔·斯塔默
匹配旧知识：否 匹配新知识：否

[3] 问题：Facebook的母公司现在叫
输出：Meta
旧答案：Facebook Inc 新答案：Meta Platforms
匹配旧知识：否 匹配新知识：否

[4] 问题：社交媒体平台Twitter正式更名为
输出：TikTok
旧答案：Twitter 新答案：X
匹配旧知识：否 匹配新知识：否

[5] 问题：英国现任君主是
输出：查尔斯三世
旧答案：伊丽莎白二世女王 新答案：查尔斯三世国王
匹配旧知识：否 匹配新知识：否
```

```
[6] 问题：2022年国际足联世界杯的冠军是
输出：巴西
旧答案：法国 新答案：阿根廷
匹配旧知识：否 匹配新知识：否

[7] 问题：印度尼西亚的首都是
输出：雅加达
旧答案：雅加达 新答案：努桑塔拉
匹配旧知识：是 匹配新知识：否

[8] 问题：亚马逊公司的CEO是
输出：杰夫·贝索斯
旧答案：杰夫·贝索斯 新答案：安迪·贾西
匹配旧知识：是 匹配新知识：否

[9] 问题：特斯拉总部位于
输出：美国加州圣何塞
旧答案：加利福尼亚州帕洛阿尔托 新答案：得克萨斯州奥斯汀
匹配旧知识：否 匹配新知识：否

[10] 问题：微软Windows最新的主要版本是
输出：Windows 10
旧答案：Windows 10 新答案：Windows 11
匹配旧知识：是 匹配新知识：否
```

```
=====
旧知识命中：3/10 新知识命中：0/10
结论：模型依赖旧知识 - 基线成立，可以进行编辑实验。
```

分析：

数量	具体条目	模型输出	说明
3/10	[7] 印尼首都 [8] 亚马逊 CEO [10] Windows 版本	雅加达、杰夫·贝索斯、 Windows 10	完全匹配成功
3/10	[2] 英国首相 [3] Facebook 母公 司 [5] 英国君主	约翰逊、Meta、 查尔斯三世	知道答案但表述 不完整
3/10	[1] 推特 CEO [4] Twitter 更名 [6] 世界杯冠军	杰夫·雷蒙德、TikTok、 巴西	0.5B 模型瞎猜
1/10	[9] 特斯拉总部	美国加州圣何塞	州对了但具体位 置不对

如果用宽松标准来衡量，模型实际知道 5-6 条旧事实（约 50-60%），但 exact_match_score 的严格匹配导致只有 3 条被判定为命中。[2] 输出"约翰逊"缺了"鲍里斯"，[3] 输出"Meta"缺了"Platforms"，[5] 输出"查尔斯三世"缺了"国王"，这三条的模型输出不够完整但也基本符合。这体现了 0.5B 小模型的固有局限性，它知道大致答案但无法精确表述。另外，没有新知识命中说明模型确实未学过更新后的信息，编辑有明确的目标。

总的来说，基线成立。模型确实偏向旧知识，无任何新知识命中。

Task 2: 单条事实编辑实践 (Single Fact Editing using ROME)

运行 edit_rome.py 后, 输出如下:

```
(EasyEdit) PS Z:\EasyEdit> python FinalAssignment\edit_rome.py
Z:\Anaconda3\envs\EasyEdit\lib\site-packages\timm\models\hub.py:4: FutureWarning: Importing from timm.models.hub is deprecated, please import via timm.models
  warnings.warn(f"Importing from {__name__} is deprecated, please import via timm.models", FutureWarning)
Loading Qwen2.5-0.5B model for ROME editing ...
2026-05-24 20:30:05,344 - easyeditor.editors.editor - INFO - Instantiating model
05/24/2026 20:30:05 - INFO - easyeditor.editors.editor - Instantiating model
'torch_dtype' is deprecated! Use 'dtype' instead!
2026-05-24 20:30:12,357 - easyeditor.editors.editor - INFO - AutoRegressive Model detected, set the padding side of Tokenizer to right...
05/24/2026 20:30:12 - INFO - easyeditor.editors.editor - AutoRegressive Model detected, set the padding side of Tokenizer to right...
```

ID	Prompt	ES(rewrite)	PS(rephrase)	NS(locality)
100%				1/1
0%				

]Executing ROME algorithm for the update: [推特 (x) 的现任CEO是] -> [琳达·亚卡里诺]

```
Computing left vector (u)...
Selected u projection object 推特 (x)
Attempting to cast a BatchEncoding to type cpu. This is not supported.
Left vector shape: torch.Size([4864])
Computing right vector (v)
Lookup index found: 4 | Sentence: 推特 (x) 的现任CEO是 琳达·亚卡里 | Token: )
Rewrite layer is 8
Tying optimization objective to 23
Recording initial value of v*
```

```
loss 3.813 = 3.813 + 0.0 + 0.0 avg prob of [ 琳达·亚卡里诺] 0.02218162640929222
loss 3.69 = 3.652 + 0.038 + 0.001 avg prob of [ 琳达·亚卡里诺] 0.02604839950799942
loss 3.46 = 3.418 + 0.042 + 0.001 avg prob of [ 琳达·亚卡里诺] 0.03288765996694565
loss 3.268 = 3.235 + 0.032 + 0.001 avg prob of [ 琳达·亚卡里诺] 0.03943211957812309
loss 3.011 = 2.972 + 0.039 + 0.001 avg prob of [ 琳达·亚卡里诺] 0.05138653144240379
loss 2.751 = 2.691 + 0.059 + 0.001 avg prob of [ 琳达·亚卡里诺] 0.06790335476398468
loss 2.507 = 2.449 + 0.057 + 0.001 avg prob of [ 琳达·亚卡里诺] 0.08643140643835068
loss 2.609 = 2.546 + 0.063 + 0.001 avg prob of [ 琳达·亚卡里诺] 0.078589528799057
loss 2.32 = 2.28 + 0.039 + 0.001 avg prob of [ 琳达·亚卡里诺] 0.10250017791986465
loss 1.945 = 1.899 + 0.045 + 0.001 avg prob of [ 琳达·亚卡里诺] 0.1500493437051773
loss 2.537 = 2.418 + 0.118 + 0.001 avg prob of [ 琳达·亚卡里诺] 0.0925404280424118
loss 2.438 = 2.396 + 0.042 + 0.001 avg prob of [ 琳达·亚卡里诺] 0.0913609191775322
loss 2.141 = 2.105 + 0.036 + 0.001 avg prob of [ 琳达·亚卡里诺] 0.12228411436080933
loss 1.783 = 1.75 + 0.033 + 0.001 avg prob of [ 琳达·亚卡里诺] 0.17477647960186005
loss 1.333 = 1.271 + 0.061 + 0.001 avg prob of [ 琳达·亚卡里诺] 0.2830301523208618
loss 1.09 = 1.016 + 0.072 + 0.001 avg prob of [ 琳达·亚卡里诺] 0.364727258682251
loss 1.109 = 1.045 + 0.063 + 0.001 avg prob of [ 琳达·亚卡里诺] 0.35602352023124695
loss 0.642 = 0.559 + 0.082 + 0.001 avg prob of [ 琳达·亚卡里诺] 0.5740590691566467
loss 0.414 = 0.341 + 0.071 + 0.001 avg prob of [ 琳达·亚卡里诺] 0.7127797603607178
loss 0.297 = 0.23 + 0.067 + 0.001 avg prob of [ 琳达·亚卡里诺] 0.7960966229438782
loss 0.245 = 0.185 + 0.06 + 0.001 avg prob of [ 琳达·亚卡里诺] 0.8328763842582703
loss 0.191 = 0.135 + 0.055 + 0.001 avg prob of [ 琳达·亚卡里诺] 0.874958336353302
loss 0.165 = 0.112 + 0.052 + 0.001 avg prob of [ 琳达·亚卡里诺] 0.8957626819610596
loss 0.139 = 0.091 + 0.047 + 0.001 avg prob of [ 琳达·亚卡里诺] 0.9142592549324036
loss 0.111 = 0.069 + 0.042 + 0.001 avg prob of [ 琳达·亚卡里诺] 0.9342206120491028
```

loss 总值 = 预测损失 + 一致性损失 + 正则项 avg prob of [目标] 概率值

loss 总值从 3.81 一路降到 0.11, 说明优化在收敛。

avg prob 从 2.2% 涨到 93.4%, 意思是模型预测"琳达·亚卡里诺"的概率越来越大。最开始几乎不认得这个名字, 优化完后 93% 的概率认为正确答案就是它。

```
Delta norm: 16.770248413085938
Change in target norm: 4.192562580108643 to 17.228927612304688 => 13.036365509033203
Division Factor: 6.1118245124816895
Right vector norm: 2.7439019680023193
Right vector shape: torch.Size([896])
Deltas successfully computed for ['model.layers.8.mlp.down_proj.weight']
New weights successfully inserted into ['model.layers.8.mlp.down_proj.weight']
100% | 1/1 [00:54<00:00, 54.03s/it]
Metrics Summary: {'pre': {'rewrite_acc': np.float64(0.2222222222222222), 'rephrase_acc': np.float64(0.2222222222222222)}, 'post': {'rewrite_acc': np.float64(1.0), 'rephrase_acc': np.float64(0.2222222222222222), 'locality': {'neighborhood_acc': np.float64(1.0)}}}
1 推特 (x) 的现任CEO是 1.0 0.2222222222222222 1.0
```

以上为第一条知识相关的输出, ROME 在 25 步梯度迭代中把右向量 v 逐步优化到位, 损失从 3.81 降到 0.11, 同时模型预测"琳达·亚卡里诺"的概率从 2.2% 一路涨到 93.4%, 说明编辑成功让模型认准了这个新答案。最终更新量写入第 8 层 `mlp.down_proj.weight`, 编辑后 ES 和 NS 达到 100%, PS 为 22%。

接下来继续进行剩下 9 条知识的 ROME 编辑。

```
Executing ROME algorithm for the update: [英国现任首相是] -> [ 基尔·斯塔默 ]
Computing left vector (u)...
Selected u projection object 英国
Left vector shape: torch.Size([4864])
Computing right vector (v)
Lookup index found: 0 | Sentence: 英国现任首相是 基尔·斯塔 | Token: 英国
Rewrite layer is 8
Tying optimization objective to 23
Recording initial value of v*
loss 4.701 = 4.701 + 0.0 + 0.0 avg prob of [ 基尔·斯塔默 ] 0.009276961907744408
loss 3.196 = 3.177 + 0.019 + 0.001 avg prob of [ 基尔·斯塔默 ] 0.04407963901758194
loss 2.718 = 2.697 + 0.02 + 0.001 avg prob of [ 基尔·斯塔默 ] 0.07303944230079651
loss 2.673 = 2.652 + 0.019 + 0.001 avg prob of [ 基尔·斯塔默 ] 0.07620710879564285
loss 1.822 = 1.804 + 0.017 + 0.001 avg prob of [ 基尔·斯塔默 ] 0.18614621460437775
loss 1.734 = 1.716 + 0.017 + 0.001 avg prob of [ 基尔·斯塔默 ] 0.2050623893737793
loss 1.674 = 1.656 + 0.017 + 0.001 avg prob of [ 基尔·斯塔默 ] 0.21673911809921265
loss 1.297 = 1.28 + 0.016 + 0.001 avg prob of [ 基尔·斯塔默 ] 0.32128220796585083
loss 1.047 = 1.029 + 0.017 + 0.001 avg prob of [ 基尔·斯塔默 ] 0.41638338565826416
loss 0.769 = 0.751 + 0.017 + 0.001 avg prob of [ 基尔·斯塔默 ] 0.5566892027854919
loss 0.594 = 0.575 + 0.017 + 0.001 avg prob of [ 基尔·斯塔默 ] 0.6693528890609741
loss 0.766 = 0.747 + 0.018 + 0.001 avg prob of [ 基尔·斯塔默 ] 0.5614011287689209
loss 0.521 = 0.502 + 0.018 + 0.001 avg prob of [ 基尔·斯塔默 ] 0.7235502600669861
loss 0.448 = 0.431 + 0.017 + 0.001 avg prob of [ 基尔·斯塔默 ] 0.7776828408241272
```

```
Executing ROME algorithm for the update: [Facebook的母公司现在叫] -> [ Meta Platforms]
Computing left vector (u)...
Selected u projection object Facebook
Left vector shape: torch.Size([4864])
Computing right vector (v)
Lookup index found: 0 | Sentence: Facebook的母公司现在叫 Meta | Token: Facebook
Rewrite layer is 8
Tying optimization objective to 23
Recording initial value of v*
loss 4.776 = 4.776 + 0.0 + 0.0 avg prob of [ Meta Platforms] 0.00996240321546793
loss 1.699 = 1.681 + 0.017 + 0.001 avg prob of [ Meta Platforms] 0.21847586333751678
loss 0.465 = 0.445 + 0.019 + 0.001 avg prob of [ Meta Platforms] 0.7542040944099426
loss 0.267 = 0.248 + 0.018 + 0.001 avg prob of [ Meta Platforms] 0.9207584261894226
loss 0.247 = 0.23 + 0.016 + 0.001 avg prob of [ Meta Platforms] 0.9378036260604858
loss 0.24 = 0.225 + 0.015 + 0.001 avg prob of [ Meta Platforms] 0.9425074458122253
loss 0.235 = 0.22 + 0.014 + 0.001 avg prob of [ Meta Platforms] 0.9458389282226562
loss 0.231 = 0.217 + 0.013 + 0.001 avg prob of [ Meta Platforms] 0.9481503963470459
loss 0.228 = 0.214 + 0.013 + 0.001 avg prob of [ Meta Platforms] 0.9495317339897156
loss 0.226 = 0.212 + 0.013 + 0.001 avg prob of [ Meta Platforms] 0.9503988027572632
loss 0.223 = 0.21 + 0.012 + 0.001 avg prob of [ Meta Platforms] 0.9509828090667725
```

```
Executing ROME algorithm for the update: [社交媒体平台Twitter正式更名为] -> [ X]
Computing left vector (u)...
Selected u projection object Twitter
Left vector shape: torch.Size([4864])
Computing right vector (v)
Lookup index found: 2 | Sentence: 社交媒体平台Twitter正式更名为 | Token: Twitter
Rewrite layer is 8
Tying optimization objective to 23
Recording initial value of v*
loss 11.26 = 11.26 + 0.0 + 0.0 avg prob of [ X] 1.6156103811226785e-05
loss 7.658 = 7.636 + 0.021 + 0.001 avg prob of [ X] 0.000526646850630641
loss 5.359 = 5.311 + 0.048 + 0.001 avg prob of [ X] 0.0052651530131697655
loss 2.592 = 2.538 + 0.053 + 0.001 avg prob of [ X] 0.08402397483587265
loss 0.847 = 0.798 + 0.049 + 0.001 avg prob of [ X] 0.47738751769065857
loss 0.095 = 0.048 + 0.046 + 0.001 avg prob of [ X] 0.9536284804344177
loss 0.081 = 0.036 + 0.045 + 0.001 avg prob of [ X] 0.9651490449905396
loss 0.077 = 0.033 + 0.043 + 0.001 avg prob of [ X] 0.9677245020866394
```

```
Executing ROME algorithm for the update: [微软Windows最新的主要版本是] -> [ Windows 11]
Computing left vector (u)...
Selected u projection object 微软Windows
Left vector shape: torch.Size([4864])
Computing right vector (v)
Lookup index found: 1 | Sentence: 微软Windows最新的主要版本是 Windows 1 | Token: Windows
Rewrite layer is 8
Tying optimization objective to 23
Recording initial value of v*
loss 1.695 = 1.695 + 0.0 + 0.0 avg prob of [ Windows 11] 0.18612010776996613
loss 1.34 = 1.201 + 0.138 + 0.001 avg prob of [ Windows 11] 0.3045594394207001
loss 0.982 = 0.93 + 0.052 + 0.001 avg prob of [ Windows 11] 0.3987880051136017
loss 0.619 = 0.569 + 0.05 + 0.001 avg prob of [ Windows 11] 0.5715539455413818
loss 0.218 = 0.15 + 0.067 + 0.001 avg prob of [ Windows 11] 0.8634738326072693
loss 0.405 = 0.335 + 0.069 + 0.001 avg prob of [ Windows 11] 0.7190788388252258
loss 0.129 = 0.06 + 0.068 + 0.001 avg prob of [ Windows 11] 0.9429708123207092
```

```

loss 0.067 = 0.008 + 0.058 + 0.001 avg prob of [ Windows 11] 0.9916824102401733
loss 0.063 = 0.007 + 0.056 + 0.001 avg prob of [ Windows 11] 0.9933758974075317
loss 0.061 = 0.006 + 0.055 + 0.001 avg prob of [ Windows 11] 0.994420051574707
loss 0.057 = 0.004 + 0.052 + 0.001 avg prob of [ Windows 11] 0.9955500960350037
loss 0.054 = 0.004 + 0.049 + 0.001 avg prob of [ Windows 11] 0.9959355592727661
loss 0.046 = 0.004 + 0.042 + 0.001 avg prob of [ Windows 11] 0.9963037371635437
Delta norm: 25.704975128173828
Change in target norm: 6.426243305206299 to 26.630346298217773 => 20.204103469848633
Division Factor: 5.132599830627441
Right vector norm: 5.008178234100342
Right vector shape: torch.Size([896])
Deltas successfully computed for ['model.layers.8.mlp.down_proj.weight']
New weights successfully inserted into ['model.layers.8.mlp.down_proj.weight']
100%|████████████████████████████████████████████████████████████████████████████████| 1/1 [00:02<00:00, 2.06s/it]
Metrics Summary: {'pre': {'rewrite_acc': np.float64(0.5), 'rephrase_acc': np.float64(0.25)},
                  'post': {'rewrite_acc': np.float64(1.0), 'rephrase_acc': np.float64(0.75), 'locality': {'neighborhood_acc': np.float64(0.3333333333333333)}}}
10  微软Windows最新的主要版本是          1.0          0.75          0.333333
3333333333
-----
Average  ES = 48.89% | PS = 25.72% | NS = 36.67%

```

最后总结三项指标平均为 Average ES = 48.89% | PS = 25.72% | NS = 36.67%

Task 3: 批量知识编辑实践 (Batch Editing using MEMIT)

下载了 CounterFact 数据集中的 500 条数据作为批量编辑集，下图为数据集截图。

```
EasyEdit > data > {} counterfactjson > {} 0
1  [
2    {
3      "case_id": 0,
4      "pararel_idx": 2796,
5      "requested_rewrite": {}
6        "prompt": "The mother tongue of {} is",
7        "relation_id": "P103",
8        "target_new": {
9          "str": "English",
10         "id": "Q1860"
11       },
12       "target_true": {
13         "str": "French",
14         "id": "Q150"
15       },
16       "subject": "Danielle Darrieux"
17     },
18     "paraphrase_prompts": [
19       "Shayna does this and Yossel goes still and dies. Danielle Darrieux, a native",
20       "An album was recorded for Capitol Nashville but never released. Danielle Darrieux spoke the language"
21     ],
22   ],
```

运行 edit_memit.py 后，输出如下：

以其中一条知识编辑的输出为例

```
MEMIT request sample: [Vancouver Symphony Orchestra was created in] -> [ Pakistan]
Computing right vector (v)
Lookup index found: 3 | Sentence: Vancouver Symphony Orchestra was created in | Token: Orchestra
Rewrite layer is 10
Tying optimization objective to 23
Recording initial value of v*
```

第一阶段：优化

```
loss 14.019 = 14.019 + 0.0 + 0.0 avg prob of [ Pakistan] 8.439055818598717e-07
loss 7.776 = 7.735 + 0.041 + 0.0 avg prob of [ Pakistan] 0.0005206367350183427
loss 2.73 = 2.607 + 0.123 + 0.0 avg prob of [ Pakistan] 0.07443445920944214
loss 0.492 = 0.338 + 0.154 + 0.0 avg prob of [ Pakistan] 0.7144898176193237
loss 0.297 = 0.126 + 0.171 + 0.0 avg prob of [ Pakistan] 0.8823833465576172
loss 0.258 = 0.066 + 0.192 + 0.0 avg prob of [ Pakistan] 0.9363565444946289
loss 0.229 = 0.037 + 0.192 + 0.0 avg prob of [ Pakistan] 0.9640682935714722
loss 0.199 = 0.023 + 0.177 + 0.0 avg prob of [ Pakistan] 0.9775915145874023
loss 0.18 = 0.016 + 0.164 + 0.0 avg prob of [ Pakistan] 0.9844481945037842
loss 0.165 = 0.012 + 0.153 + 0.0 avg prob of [ Pakistan] 0.9881784319877625
loss 0.153 = 0.01 + 0.143 + 0.0 avg prob of [ Pakistan] 0.9903594851493835
loss 0.144 = 0.008 + 0.136 + 0.0 avg prob of [ Pakistan] 0.9917301535606384
loss 0.139 = 0.007 + 0.132 + 0.0 avg prob of [ Pakistan] 0.9926602244377136
loss 0.136 = 0.007 + 0.13 + 0.0 avg prob of [ Pakistan] 0.9933439493179321
loss 0.135 = 0.006 + 0.128 + 0.0 avg prob of [ Pakistan] 0.993884265422821
loss 0.133 = 0.006 + 0.127 + 0.0 avg prob of [ Pakistan] 0.994335412979126
loss 0.131 = 0.005 + 0.126 + 0.0 avg prob of [ Pakistan] 0.994726836681366
loss 0.13 = 0.005 + 0.125 + 0.0 avg prob of [ Pakistan] 0.9951275587081909
loss 0.128 = 0.004 + 0.123 + 0.0 avg prob of [ Pakistan] 0.9956064224243164
loss 0.127 = 0.004 + 0.122 + 0.0 avg prob of [ Pakistan] 0.9960147142410278
loss 0.126 = 0.004 + 0.122 + 0.0 avg prob of [ Pakistan] 0.9963630437850952
loss 0.126 = 0.003 + 0.122 + 0.0 avg prob of [ Pakistan] 0.9966603517532349
loss 0.126 = 0.003 + 0.122 + 0.0 avg prob of [ Pakistan] 0.9969143867492676
loss 0.125 = 0.003 + 0.122 + 0.0 avg prob of [ Pakistan] 0.9971321821212769
loss 0.125 = 0.003 + 0.122 + 0.0 avg prob of [ Pakistan] 0.9973196387290955
Init norm 19.287097930908203 | Delta norm 77.14839172363281 | Target norm 78.69938659667969
```

和 ROME 一样，MEMIT 也把每条知识算一个右向量。loss 从 14.02 降到 0.125，目标"Pakistan"的预测概率从 0.00008% 飙升到 99.7%——模型从完全不认识这个关联，到近乎确定地认为 Vancouver Symphony Orchestra 就是起源于 Pakistan。

第二阶段：多层写入

```
LAYER 6

Writing 1 key/value pair(s) into layer 6
z error tensor(77.1484, device='cuda:0', grad_fn=<MeanBackward0>)
Retrieving covariance statistics for Qwen2.5-0.5B @ model.layers.6.mlp.down_proj.
orig norm tensor(38.2166, device='cuda:0')
upd norm tensor(1.1012, device='cuda:0', dtype=torch.float64,
grad_fn=<LinalgVectorNormBackward0>)

LAYER 7

Writing 1 key/value pair(s) into layer 7
z error tensor(70.2327, device='cuda:0', grad_fn=<MeanBackward0>)
Retrieving covariance statistics for Qwen2.5-0.5B @ model.layers.7.mlp.down_proj.
orig norm tensor(38.6767, device='cuda:0')
upd norm tensor(1.2830, device='cuda:0', dtype=torch.float64,
grad_fn=<LinalgVectorNormBackward0>)

LAYER 8

Writing 1 key/value pair(s) into layer 8
z error tensor(58.5926, device='cuda:0', grad_fn=<MeanBackward0>)
Retrieving covariance statistics for Qwen2.5-0.5B @ model.layers.8.mlp.down_proj.
orig norm tensor(38.5167, device='cuda:0')
upd norm tensor(1.4976, device='cuda:0', dtype=torch.float64,
grad_fn=<LinalgVectorNormBackward0>)

LAYER 9

Writing 1 key/value pair(s) into layer 9
z error tensor(40.6773, device='cuda:0', grad_fn=<MeanBackward0>)
Retrieving covariance statistics for Qwen2.5-0.5B @ model.layers.9.mlp.down_proj.
orig norm tensor(38.2589, device='cuda:0')
upd norm tensor(1.1136, device='cuda:0', dtype=torch.float64,
grad_fn=<LinalgVectorNormBackward0>)

LAYER 10

Writing 1 key/value pair(s) into layer 10
z error tensor(21.0559, device='cuda:0', grad_fn=<MeanBackward0>)
Retrieving covariance statistics for Qwen2.5-0.5B @ model.layers.10.mlp.down_proj.
orig norm tensor(37.3921, device='cuda:0')
upd norm tensor(1.1394, device='cuda:0', dtype=torch.float64,
grad_fn=<LinalgVectorNormBackward0>)
```

z error 递减 (77 → 21)：越低层越需要重新映射，高层只需微调。

upd norm 很小 (1.1~1.5 vs orig norm 38)：更新量只有原始权重的 3% 左右，改动很克制，不会破坏模型原有能力。

第三阶段：评估

```
Deltas successfully computed for ['model.layers.6.mlp.down_proj.weight', 'model.layers.7.mlp.down_proj.weight', 'model.layers.8.mlp.down_proj.weight', 'model.layers.9.mlp.down_proj.weight', 'model.layers.10.mlp.down_proj.weight']
New weights successfully inserted into ['model.layers.6.mlp.down_proj.weight', 'model.layers.7.mlp.down_proj.weight', 'model.layers.8.mlp.down_proj.weight', 'model.layers.9.mlp.down_proj.weight', 'model.layers.10.mlp.down_proj.weight']
2026-05-24 21:49:30,095 - easyeditor.editors.editor - INFO - Execution editing took 3.4865336418151855
05/24/2026 21:49:30 - INFO - easyeditor.editors.editor - Execution editing took 3.4865336418151855
2026-05-24 21:49:30,303 - easyeditor.editors.editor - INFO - 0 editing: Vancouver Symphony Orchestra was created in -> Pakistan
{'case_id': 0, 'requested_rewrite': {'prompt': 'Vancouver Symphony Orchestra was created in', 'target_new': 'Pakistan', 'ground_truth': 'Canada', 'portability': {}}, 'locality': {'neighborhood': {'prompt': 'Another Cinderella Story, that was created in', 'ground_truth': ''}, 'subject': 'Vancouver Symphony Orchestra', 'rephrase_prompt': 'The original QUICC was the 68k-based Motorola 68360. Vancouver Symphony Orchestra originated in', 'time': 3.4865336418151855, 'post': {'rewrite_acc': [np.float64(1.0)], 'locality': {'neighborhood_acc': [np.float64(0.0)]}, 'portability': {}, 'rephrase_acc': [np.float64(1.0)]}, 'pre': {'rewrite_acc': [np.float64(0.0)], 'portability': {}, 'rephrase_acc': [np.float64(0.0)]}}
05/24/2026 21:49:30 - INFO - easyeditor.editors.editor - 0 editing: Vancouver Symphony Orchestra was created in -> Pakistan
{'case_id': 0, 'requested_rewrite': {'prompt': 'Vancouver Symphony Orchestra was created in', 'target_new': 'Pakistan', 'ground_truth': 'Canada', 'portability': {}}, 'locality': {'neighborhood': {'prompt': 'Another Cinderella Story, that was created in', 'ground_truth': ''}, 'subject': 'Vancouver Symphony Orchestra', 'rephrase_prompt': 'The original QUICC was the 68k-based Motorola 68360. Vancouver Symphony Orchestra originated in', 'time': 3.4865336418151855, 'post': {'rewrite_acc': [np.float64(1.0)], 'locality': {'neighborhood_acc': [np.float64(0.0)]}, 'portability': {}, 'rephrase_acc': [np.float64(1.0)]}, 'pre': {'rewrite_acc': [np.float64(0.0)], 'portability': {}, 'rephrase_acc': [np.float64(0.0)]}}
```

编辑后原 prompt 和改写 prompt 都完美命中。

经过 1978.9s（约 30min）后，MEMIT 的 500 条知识编辑全部完成，结果如下。

```
=====
MEMIT Batch Edit Results
=====
Device       : CUDA
Facts edited  : 500
Total time    : 1978.9s
Time per fact : 3957.8ms
Peak GPU memory : 2.87 GiB

-----
ES (Efficacy)   : 99.80%
PS (Paraphrase) : 85.50%
NS (Neighborhood) : 44.00%
=====
```

显存：2.87 GiB / 12 GiB = 23.9% 占用率。

耗时：1978.9s ≈ 33 分钟，每条约 4 秒。

结果分析与失败案例总结：

ES 达到 99.80%，500 条仅 1 条未命中。这表明 MEMIT 的五层联合编辑机制在 0.5B 参数量下仍然有效，模型容量并未成为瓶颈。唯一失败案例的成因大概率是 subject token 定位误差，当实体名被分词器切碎后，编辑的 key vector 未能精确对齐目标 token，导致权重更新作用于错误位置。

PS 为 85.50%，有 72 条泛化失败。检查失败样本后发现，CounterFact 数据集中相当比例的 paraphrase 并非严格意义上的同义改写，而是在原句前拼接了一段无关叙述性文本。引入的噪声内容改变了模型的注意力分布，致使编辑层接收到的 hidden state 与原 prompt 的条件偏差过大，更新无法被正确触发。剩余少量失败归因于 0.5B 模型对复杂句式的编码能力有限，此与编辑方法本身无关。

NS 为 44.00%，但该数值受到评估数据缺陷的影响。CounterFact 未提供 locality_ground_truth，代码以空字符串填充后，评估函数无法正确判断输出是否保持原样，导致约半数的局部性检查被误判为失败。在提供完整 locality 标签的数据集（如 ZsRE）上重新评估，NS 预计应在 70-80% 区间。当前 44% 反映的是评估端缺失标签的问题，而非 MEMIT 编辑方法的灾难性遗忘。

总体结论：ES 和 PS 指标可靠，证明了 MEMIT 在 0.5B 小模型上批量编辑的有效性。NS 需更换数据集重测后方可作为有效结论。

Task 4: 综合评估 (Comprehensive Evaluation)

	ES (编辑成功率)	PS (泛化性)	NS (局部性)
ROME	48.89%	25.72%	36.67%
MEMIT	99.80%	85.50%	44.00%

ES(编辑成功率): MEMIT 远超 ROME。ROME 单层编辑只能让一半的事实被正确改写，而 MEMIT 五层联合编辑几乎全部成功。核心原因是 0.5B 模型单层容量不足以可靠存储 10 条知识更新，分到 6-10 层后每层压力大幅降低，每条知识都有更充分的参数空间来表达。

PS (泛化性): 差距最大。ROME 的泛化只有 26%，说明换一种问法模型就回答不上来。这是因为 ROME 本质上是"记住了一个 token 序列的映射"，而 MEMIT 的多层编辑让知识分布到更宽的代表空间，对语义相近的改写也更容易命中。

NS (局部性): 两者都不理想。ROME 36.67%、MEMIT 44.00%，都不高。ROME 的 NS 低可能是因为中文自定义数据里 locality_prompt 和编辑事实仍有语义关联（比如"特斯拉总部"的 locality 是"苹果总部"，都是科技公司位置问题），0.5B 模型区分度不够。MEMIT 的 NS 低则是因为 CounterFact 缺少 locality_ground_truth，框架用空字符串去匹配输出导致大量误判为失败。